INTELLIGENT SYSTEM (BICS 2303)

SEMESTER 2 2025/2026

TOPIC:

SIGN LANGUAGE DETECTION USING IMAGES

LECTURER NAME:

ASST. PROF. DR. DINI HANDAYANI

GROUP NAME:

YOLO

GROUP MEMBERS:

NO	NAME	MATRIC NO.
1	CHE NURUL LIYANA BINTI CHE MAT NOOR	2317600
2	DAYANA BATRISYA BINTI RAMILE	2318872
3	NUR NAJIHAH BINTI MOHD ZAMRISHAM	2319880
4	NURUL HANA BINTI ABDUL HALIM	2315932
5	NURUL IZZAH BINTI ZUBLI	2414054

TABLES OF CONTENTS

1.0 INTRODUCTION 4

1.1. Background 4

1.2. Problem Statement 4

1.3. Objectives 5

1.4. Scope of Project 5

2.0 LITERATURE REVIEW 7

2.1. Related Studies 7

2.2. Existing Solutions 8

2.3. Comparison of Methods 9

3.0 METHODOLOGY 11

3.1.Dataset Description 11

3.1.1.YOLOv5 Regional Alphanumeric 11

3.1.2.MediaPipe ASL Dataset 12

3.2.Preprocessing Techniques 12

3.2.1.Resolution Downscaling and Spatial Aspect Ratio Anchoring 12

3.2.2.Color-Channel Standardization and Filtering 12

3.2.3.Annotation Syntax and Spatial Coordinate Validation 13

3.2.4.Configuration Path Mapping and Transfer Learning Weights 13

3.3.ML Algorithms Used 13

3.3.1.YOLOv5s (Pixel-Based CNN) 13

3.3.2.MediaPipe Hands & DNN (Landmark-Based) 14

3.4.System Architecture/Framework 14

3.4.1.YOLOv5 System Architecture 14

3.4.2.YOLOv5 Working Principle & Loss Function 14

3.4.3.YOLOv5 System Framework 15

3.4.4.MediaPipe Framework 16

4.0 SYSTEM DEVELOPMENT 16

4.1. Workflow/Process 16

4.2. Implementation Details 17

4.3. Screenshots of the System 19
5.0 EXPERIMENTAL RESULTS 20
5.1. Training/Testing Results 20
5.1.1. YOLOv5s 20
5.1.2. MediaPipe Model 20
5.2. Performance Comparison 20
5.3. Evaluation Metric 22
5.4. Analysis and Discussion 22
5.4.1 Analysis of Four Distinct Training Data Scales 22
5.4.2. Analysis of The Evaluation Result 25
5.4.3. Discussion on YOLOv5s Training Model 26
5.4.4. Comparative Analysis of Object Detection of YOLOv5s and Skeletal Mesh of MediaPipe 26
6.0 CONCLUSION 27
6.1. Key Findings: 27
6.2. Challenges Faced 29
6.3. Future Improvements 30
6.4. Contribution of Each Member 32
7.0 REFERENCES 33

1.0 INTRODUCTION

1.1. Background

Communication is a fundamental human need, but there exist individuals with hearing and speech impairments facing difficulties communicating in a predominantly hearing community. Thus, the medium for communication for people with hearing disability is sign language. It requires visual communication using hand gestures, facial expressions, and body language to convey what they want to say. Like a language that is diverse, there are over 300 different sign languages used worldwide depending on its country, language, or communities such as American Sign Language (ASL). Learning sign language is important because it can break communication barriers between people with normal hearing and deaf or mute individuals. It processes clear and efficient communication to avoid any misunderstandings.

Recent advancement technology in machine learning has encouraged the stability of automated Sign Language Recognition (SLR) systems. This project developed **two recognition prototypes**: a YOLOv5-based model that performs hand gesture recognition using raw image features and a MediaPipe Hands-based model that utilizes skeletal landmark extraction for sign language recognition. Modern systems improve performance by extracting key features from human hand to create reliable spatial representations (León-Quintana et al., 2026). By comparing these two prototypes, able to evaluate their differences in computational efficiency, higher accuracy, and better adaptive systems. This advance of technology can integrate into user-friendly applications to help reduce communication gaps between two communities.

1.2. Problem Statement

The main issues of this project address the communication gap between deaf community and hearing individuals in daily environments. While deep learning offers automated sign language translation, several challenges must be resolves to develop a viable system:

1. **Sensitivity to Background Noise and Clutter**

   The system relies on raw image data, meaning the image can have severe distortions like poor lighting, complex background, or user stand so far from the camera making it difficult to recognize the gesture. Therefore, the systems needed to accurately isolate hand track movements regardless of environmental issues.

2. **Difficulty Recognizing Similar Signs**

   Traditional classification methods may fail to properly differentiate between these signs since sign language gestures have similar hand shapes but hold a different meaning. These signs can be

confusing which lead to misleading communication between individuals. Hence, the systems are required to reduce the probability of error in recognizing small differences between gestures.

3. **Lack of Algorithm Performance Comparison**

   Using machine learning techniques does not provide enough insight into the balance between accuracy and processing speed. To discover the most effective model for real-time sign language recognition, the methods must be compared and evaluate their differences against alternative non-linear machine learning topologies.

1.3. Objectives

The objectives of this project are:

- To gain a better understanding of Machine Learning and Supervised Learning concepts.
- To develop a YOLOv5 Convolutional Neural Network (CNN) model that can classify hand gestures representing the alphabet (A–Z) and digits (1–9).
- To develop a secondary prototype using MediaPipe Hands to extract skeletal landmarks for sign language recognition and evaluate its effectiveness as an alternative to raw pixel-based recognition.
- To compare the performance of the YOLOv5 and MediaPipe Hands approaches in terms of recognition accuracy, computational efficiency, and robustness under different environmental conditions.

1.4. Scope of Project

The project scoped is defined to make sure that project workflow is achievable within the timelines, as follows:

- **Target Domain and Dataset**: The project focused on Sign Language Recognition (SLR) using dataset of distinct sign language characters, which are alphabets from A to Z and digits from zero to nine. The primary YOLOv5 model is trained and evaluated using a curated alphanumeric hand gesture dataset, while the MediaPipe Hands prototype utilizes a separate American Sign Language (ASL) landmark-based dataset to evaluate cross-dataset robustness and the adaptability of the proposed recognition architectures. This implementation did not include recognition of sentences, facial expressions, and emotions.

- **Algorithm and Model Development:** This project uses supervised learning techniques to recognize gestures of sign language. The hand landmark is applied to decrease the impact of background condition and focal loss to improve the training efficiency and classification of similar gestures of hand.
- **System Validation:** The system's performance will be validated thoroughly using Precision, Recall, and mean Average Precision (mAP).

2.0 LITERATURE REVIEW

2.1. Related Studies

You Only Look Once version 5 (YOLOv5) is an effective algorithm in recognition of sign language. In YOLOv5 family, there exist many versions such as n, s, m, l, and x with the difference in width and depth. However, YOLOv5s are highly recognized as lighter versions among other versions. According to Jing et al. (2025), YOLOv5 ideally qualifies for mobile app use, as the design to be fast and light to execute on phone processor and battery. In Sign Language Recognition (SLR), YOLOv5 anchor-based object detector delivers high performance as the research from Ningsih et al. (2024), show that YOLOv5s successfully deliver solid ability to SLR with mean Average Precision (mAP) of 97.9% and a recall of 99.7%.

In recent literature, it underscored a few technological optimizations for interpreting gestures. The first optimization is attention to mechanisms that occur when the Convolutional Block Attention Module (CBAM) has been added to algorithm neck part showing improvement in accuracy recognizing the image of sign language (Jing et al., 2025). The CBAM helps the model focus on important image pixels, alongside reducing model complexity. Next, the technological optimizations are refinement of loss function where it Upgrade bounding box loss to Focal Complete Intersection over Union (Focal CIoU) helps the system to give maximum attention to difficult classification images, therefore reduce missed defection (Jing et al., 2025) and lastly the data augmentation prove the reliability of the YOLOv5 system is attributed to thorough data augmentation techniques like horizontal flip, rotation, saturation and brightness adjustments (Ningsih et al., 2024) simulate real-world environments.

In contrast with YOLOv5, Media Pipe tracks skeletal key points representing human joints from static or moving image based on pixel grids, transforming high dimensional video into a lighweight, non-linear coordinate representation that working effectively with human motion (Tabari et al., 2018). The **Media Pipe Holistic** framework developed by Google Research, has a comprehensive extraction tool such as extraction a total of 543 anatomical landmarks, consisting of 33 pose landmarks for upper body, 21 for each hand (42 for both hands), and 468 facial landmarks. These landmarks provide details of spatial coordinates representing positions of key body joints. Next, the component of this framework is capturing natural anatomical relationships between joints. Media Pipe moves beyond pixel grids' limitations to capture the natural connectivity and structural cues for distinguishing different sign movements. The last fundamental component is these landmarks processed frame-by-frame to form an aggregated temporal sequence. This enables models to learn motion trajectories and dynamic gesture patterns across continuous frames, which are important for accurate sign language recognition.

2.2. Existing Solutions

Spatial-Temporal Graph Convolutional Network (ST-GCN) established first model to graph-based neural networks to skeletal data. As reported by Yan et al. (2018), ST-GCN model topology is structured in three key elements:

- **Joints as Nodes:** Each skeletal landmark, for instance, finger joint and facial point is denoted as node in the graph
- **Spatial Edges:** Nodes and edges connected to each other match the natural connectivity of the human body.
- **Temporal Edges:** The same nodes (joints) connected across uninterrupted video frames to track the trajectory of motion over time.

Graph based architecture offer advantages for isolated joint sign detection after moving forward beyond the limitation of traditional convolutional networks:

- **Implicit Feature Learning:** ST-GCN automatically learns spatial and temporal feature from data, in contrast with earlier model that relied on hand crafted part or manual traversal rules (Yan et al., 2018)
- **Hierarchical Representation:** The network allows the model to captured joints movement in body parts, such as individual finger or hands, providing much greater expressive power compare with raw pixel analysis (Yan et al., 2018)
- **Enhanced Generalization:** ST-GCN demonstrate stronger generalization capabilities across different signers and environments because Graph Neural Networks (GNNs) only focus on the relationships between the joints (Yan et al., 2018).

The component in non-linear topologies involving Recurrent Neural Networks (RNNs) and Attention mechanism to process temporary sequences of skeletal landmarks. While GNNs address spatial detection and structural of human hand relationships, RNNs address that “sign language is inherently temporal and therefore requires sequential modelling, playing a central role in recognition” (Tabari et al., 2018). Sequential networks like **Long Short-Term Memory (LSTM)** networks, particularly Bidirectional LSTMs (BiLTSM) are often used to model progression of sign over time. As reported by Tabari et al. (2018), these networks architectures effectively learn to handle vanishing gradient problems found in standard RNNs the entire time of sign gesture. By processing data sequences in forward and backward directions, the BiLTSM network is important for differentiating signs with overlapping structural trajectories.

The addition of an attention layer to the LTSM optimizes this topology by allowing the model to dynamically weight temporal segments. The attention mechanism calculates an attention score that heavily focuses on performative portions of a sign while assigning less weight to noisy and unnecessary frames. As stated in Tabari et al. (2018), this improvement provides two important benefits: robustness and interpretability. For robustness, it provides the attention layer minimizes the impact of background variation and signer-specific minor movements that unrelated to actual sign semantics and interpretability increases the model to focus on critical temporal regions like peak of hand shape leading to accuracy tests as high as 94.69% for the attention-enhanced LSTM framework.

2.3. Comparison of Methods

The final literature review shows clear trade-off between three distinct topologies, which is Anchor-based (YOLOv5), Media Pipe Holistic, Non-linear Graph (ST-GCN), and Sequential and Attention mechanism (LSTM). This comparison points out the trade-off between computational efficiency, performance capabilities, and environmental robustness.

Table 2.1: Comparative Summary of Methods Table

Method	YOLOv5	MediaPipe Holistic	ST-GCN	LSTM
Type	Anchor-based Object Detector	Pose-Estimation Framework	Non-linear Graph Topology	Sequential with Attention
Computational Efficiency	Operates on raw pixel grids can be computationally intensive and requires higher power resources	Enables the dimensionality reduction by converting video frames into 543 key points	Processes only isolated skeletal make it more efficient than image-based methods	Produce lightweight landmark sequence for low-resource settings and mobile hardware
Environmental Robustness	Highly sensitive to background conditions.	Highly robust to background variations because it isolates skeletal key points from image background	Learning pattern directly from connected join, improving robustness and stronger generalization across different signers	Uses an attention mechanism to emphasize motion patterns, filtering out irrelevant movement.
Performance Capabilities	Excels in static sign recognition, achieving mean Average Precision (mAP) of 97.2% to 98.12%	Accurate real time landmark extraction which provides reliable input features for recognition models	Well suited for action-based recognition, treating modelling video as spatial-temporal graph.	Recognition accuracy up to 94.69% by capturing long-term temporal dependencies.

3.0 METHODOLOGY

3.1. Dataset Description

3.1.1. YOLOv5 Regional Alphanumeric

The dataset used in this project is specifically curated for static Sign Language Recognition (SLR) focusing on regional alphanumeric characters. It contains distinct visual gesture representations encompassing the complete English alphabet (A to Z) and single-digit integers (1 to 9), establishing a comprehensive sign vocabulary

To optimize performance and eliminate virtual memory overload errors within the cloud training environment, an empirical sample-tuning strategy was conducted via an automated script in Step 3. While the global Kaggle source data repository holds more than 40,000 baseline image profiles, loading this complete dataset simultaneously overwhelmed local system environments and triggered critical hardware crashes on the T4 GPU.

Therefore, a programmatic Stratified Sampling routine was embedded into the data pipeline. The script iterates through each alphabetical and numerical class directory, executing a random selection filter (random.sample()) to extract exactly 100 images per class. This scales the active dataset down to an optimized, highly expressive threshold subset of approximately 2,000 images. This localized volume provides sufficient structural and intra-class variance including minor shifts in finger-joint occlusion and hand orientation without exceeding system hardware processing bounds.

The finalized 2,000-image dataset was partitioned dynamically using an automated 80/20 ratio split per class to preserve strict class balance across the entire vocabulary:

- Training Set (80%): Exactly 80 images per class (totaling 1,600 images) are directed to the training data directory to actively update the model parameters and weight profiles.
- Validation Set (20%): Exactly 20 images per class (totaling 400 images) are directed to the validation data directory to test cross-entropy optimization stability and evaluate convergence curves on unseen instances.

3.1.2. MediaPipe ASL Dataset

American Sign language (ASL) is our secondary dataset that was compiled from a specialized Kaggle sign language repository encompassing alphanumeric classes (letters 'A' through 'Z' and digits '1' through '9'). To ensure absolute class balance, a stratified sampling pipeline was implemented, selecting a uniform distribution of images per class to establish a standardized baseline.

For benchmarking rationalization, a separate dataset was explicitly used to benchmark the MediaPipe implementation against the baseline YOLOv5s model. By evaluating both frameworks on an isolated, independent data split, the study ensures a fair cross-domain comparison. This isolates the true performance variance between landmark-based tracking and raw pixel-based object detection without any risks of data leakage.

3.2. Preprocessing Techniques

To move beyond default platform configurations and maximize feature extraction precision on our single-pass network, the raw image stream was subjected to a four-tier programmatic preprocessing pipeline written in Python.

3.2.1. Resolution Downscaling and Spatial Aspect Ratio Anchoring

Raw images varied drastically in original pixel density, posing a structural risk of high computational loads and sudden hardware timeouts. As executed in Step 3, a custom image processing script programmatically resized all incoming image matrices to a uniform resolution of 416 × 416 pixels. This spatial resolution acts as an optimization threshold; it drastically lowers the processing load compared to standard high-resolution images, thereby preventing runtime failures on the T4 GPU while preserving vital bounding box regression priors relative to the internal anchor grid intersections.

3.2.2. Color-Channel Standardization and Filtering

To ensure consistent illumination properties across varying collection backgrounds, color-channel filtering was embedded into the loader script. All source images were forced into a standardized 3-channel RGB (Red-Green-Blue) format. Pixel intensity matrices were mathematically normalized by dividing values by 255 to map them into a $[0, 1]$ float space, effectively removing extreme exposure spikes and stabilizing baseline gradient calculations in the early convolutional filters.

3.2.3. Annotation Syntax and Spatial Coordinate Validation

The dataset relied on external character annotations. Before entering the neural network, the automated script in Step 3 handled syntax validation to prevent structural errors from corrupting the bounding box coordinates. The script reads each text entry, writing the standard object detection bounding box coordinate format required by YOLOv5: [Class ID] [X_center] [Y_center] [Width] [Height].

Normalized bounding coordinates were verified to be strictly bounded within [0.0, 1.0]. Any orphaned images lacking valid annotations or containing corrupted text strings were excluded automatically, ensuring perfect data alignment with the YAML configuration maps.

3.2.4. Configuration Path Mapping and Transfer Learning Weights

Once the subset directory structures were fully established, Step 4 executed a path-mapping script to scan the active files and generate a single unified configuration text layout (train_subset.txt). This maps the exact file locations of the 2,000 target subset images so the train.py script can locate data records seamlessly during runtime.

To minimize overall training times, transfer learning was implemented by utilizing initial weights trained on the Microsoft Common Objects in Context (MS COCO) dataset (yolov5s.pt). The network maps low-level visual features—such as edge detection filters, horizontal boundary alignments, and texture gradients learned from everyday objects—directly onto the anatomical structures and joint vectors of human hands. Leveraging these pre-initialized general features allowed the model to bypass random initializations, accelerate gradient convergence, and achieve a highly stable configuration within our specific target sign language domain.

3.3. ML Algorithms Used

To evaluate different paradigms of sign language recognition, this project utilizes two distinct machine learning methodologies: a pixel-based object detector (YOLOv5s) and a landmark-based structural classifier (MediaPipe Hands paired with a Deep Neural Network).

3.3.1. YOLOv5s (Pixel-Based CNN)

YOLOv5s is a single-stage, anchor-based Convolutional Neural Network (CNN) engineered for real-time object detection. Unlike traditional networks that only perform classification, YOLOv5 divides

an image into an $S \times S$ grid to predict bounding boxes (localization) and class probabilities simultaneously in a single pass.

For this project, YOLOv5s (Small) was selected from the Ultralytics family as the baseline architecture. Incorporating approximately 7.2 million parameters, it strikes an optimal balance between accuracy and computational efficiency, ensuring fluid real-time performance and seamless deployment within resource-constrained environments like Google Colab.

3.3.2. MediaPipe Hands & DNN (Landmark-Based)

In contrast to the pixel-dependent approach of YOLOv5s, MediaPipe completely abstracts the hand from its surrounding environment. It isolates the target structure by stripping away background clutter, illumination changes, and skin-tone variations.

The framework achieves this by mapping a structural landmark mesh of 21 3D coordinates (X, Y, Z) tracking from the wrist up to the fingertips. This pipeline collapses high-dimensional raw image data into a lightweight, compact geometric coordinate vector, which is then fed into a sequential Deep Neural Network (DNN) for final gesture classification.

3.4. System Architecture/Framework

3.4.1. YOLOv5 System Architecture

The architecture of YOLOv5 (which apply to all YOLOv5 versions including YOLOv5s) is divided into three core mathematical components:

- **The Backbone:** This acts as the feature extractor. It uses Cross Stage Partial (CSP) networks to reduce redundant gradient information. Mathematically, it simplifies the data while preserving the most important features of a hand gesture.
- **The Neck:** This layer handles feature aggregation. It combines information from different scales of the image, ensuring the model can detect a hand whether it is close to the camera or far away.
- **The Head:** This is the prediction engine. It divides the input image into a grid and calculates the probability of a sign language character being present in each cell.

3.4.2. YOLOv5 Working Principle & Loss Function

YOLOv5 treats detection as a regression problem. It takes the entire image as an input and passes it through the network to predict:

- **Bounding Boxes (x, y, w, h):** The coordinates and size of the detected hand.
- **Confidence Score:** The confidence that a hand actually exists in that box.
- **Class Probabilities:** The probability that the detected hand represents a specific sign.

To iteratively optimize network weights during the learning process, the framework evaluates model predictions against ground-truth annotations using a multi-task composite Loss Function:

$$\text{Loss} = \lambda_1 \cdot L_{cls} + \lambda_2 \cdot L_{obj} + \lambda_3 \cdot L_{loc}$$

- L_{cls} **(Classification Loss):** Uses Binary Cross-Entropy to penalize wrong sign identification.
- L_{obj} **(Objectness Loss):** Penalizes the model for missing a hand.
- L_{loc} **(Localization Loss):** Uses CIoU (Complete Intersection over Union) to measure how accurately the predicted box overlaps with the actual hand coordinates.

3.4.3. YOLOv5 System Framework

The operational framework of YOLOv5 is structured as a sequential, four-tier pipeline: Data Sampling & Partitioning, YAML Configuration Mapping, Core Deep Learning and Evaluation.

Data Sampling & Partitioning: The input boundary of the framework handles raw data streams. It dynamically processes incoming hand gesture images while isolating uniform subsets for training and validation.

YAML Configuration Mapping: Before features can be extracted, the framework maps data properties to model requirements. This ensures that the neural network's final output layer is perfectly aligned with the exact dimensions of the target sign language vocabulary.

Core Deep Learning: Once configurations are initialized, the data enters the primary processing layer. This engine utilizes YOLOv5s architecture by focusing on extracting high-level spatial features specific to human hand morphology through its interconnected Backbone, Neck, and Head sub-frameworks.

Evaluation: The system applies a confidence filter threshold to eliminate background noise and low probability predictions. The final output generates spatial bounding boxes and categorical probability scores to deliver real-time sign language identification.

3.4.4. MediaPipe Framework

The MediaPipe framework redefines the feature extraction pipeline by prioritizing geometric joint structures over raw 2D pixel grids.

Pixel-to-Coordinate Translation: Standard convolutional models must read every pixel across an image matrix to detect a gesture. The MediaPipe pipeline bypasses this by running an internal single-shot palm detector. Once the hand is localized, it discards the rest of the image pixels entirely and focuses solely on extracting the spatial vectors of the 21 key joints.

Data Streamlining: By tracking the relative positions, distances, and angles between the joint coordinates rather than analyzing 2D pixel matrices, the system drastically cuts down computational overhead. The input transitions from a massive pixel array to a lightweight, normalized 1D vector of numerical coordinates, transforming a complex image processing problem into an efficient numerical classification task.

4.0 SYSTEM DEVELOPMENT

The development workflow of the project is extended to accommodate our multi-prototype approach, separating the anchor-based object detection framework from the structural landmark pipeline. By incorporating a secondary pipeline, the system transitions from a pure raw-pixel bounding box regression model to a localized geometric feature space. The structural execution flows, core programmatic configurations, and visual interface environments for both concurrent prototypes are thoroughly detailed below.

4.1. Workflow/Process



Figure 4.1: System execution flow and real-time inference pipeline for YOLOv5.



Figure 4.2: System execution flow and real-time inference pipeline for Mediapipe.

Unlike the primary YOLOv5 paradigm which executes full-image bounding box regressions across a raw pixel grid, this parallel pipeline isolates spatial processing into two independent phases. MediaPipe handles real-time anatomical tracking by converting visual structures into lightweight mathematical coordinate data, discarding irrelevant environmental variations.

Once the hand landmarks are isolated, the extracted vector array is routed directly into a Multi-Layer Perceptron (MLP) engine which focuses exclusively on gesture classification. This functional segregation reduces total computational overhead and prevents common detection faults caused by complex backgrounds. The classified gesture is then processed through an integrated text-generation script that appends characters into alphanumeric sentences and manages punctuation dynamics via distinct physical command triggers (such as SPACE and DELETE actions).

4.2. Implementation Details

To execute the two parallel sign language recognition pipelines, distinct backend scripts and command-line arguments are implemented within the application environment.

YOLOv5 Prototype Run Configurations

The anchor-based model executes real-time inference via the detect.py module using the following terminal configuration parameters:

- **--weights runs/train/exp/weights/best.pt:** Instructs the execution engine to pass the specific, custom-learned spatial and semantic features from the model training phase directly into the active application workspace.
- **--img 416:** Configures the automated downscaling engine to uniformly standardize any input user media to a resolution matrix of 416×416 pixels, matching the dimensions used during training.
- **--conf 0.25:** Instructs the system's output filtering logic to display only predicted objects where the model's geometric and class confidence meets or exceeds a 25% threshold.
- **--source 0:** Establishes the programmatic data stream pointer, routing input data directly from the system’s primary video hardware or evaluation folders into the model's processing layer.

MediaPipe & MLP Prototype Implementation

The secondary landmark-based prototype utilizes an interactive Python script leveraging the OpenCV, MediaPipe, and TensorFlow libraries. Below is the localized code snippet demonstrating the core loop used to extract 3D joint coordinates, format the data, and perform real-time structural inference:

```
    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            mp_draw.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)

            landmarks = []
            for lm in hand_landmarks.landmark:
                landmarks.extend([lm.x, lm.y, lm.z])

            input_data = np.array(landmarks).flatten().reshape(1, -1)
            prediction = mlp_model.predict(input_data, verbose=0)
            class_id = np.argmax(prediction)
            confidence = np.max(prediction)

            label = LABELS[class_id] if class_id < len(LABELS) else str(class_id)
            text = f"{label} ({confidence*100:.1f}%)"
            cv2.putText(frame, text, (10, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

    cv2.imshow("Sign Language Recognition System", frame)
```

This structural configuration provides direct performance enhancements over raw pixel processing through several distinct parameters:

- **3D Spatial Feature Extraction (lm.x, lm.y, lm.z):** Instead of relying purely on 2D coordinates, the system extracts depth estimations (z-axis) alongside width and height parameters. This rich, multi-dimensional feature set provides the Multi-Layer Perceptron (MLP) with a highly detailed representation of hand morphology.
- **Data Reshaping (flatten().reshape(1, -1)):** The extracted landmark coordinates are programmatically flattened into a continuous 1D tensor array. This exact structuring is required to align the raw data directly with the dense input layers of the TensorFlow/Keras neural network architecture.
- **Confidence Scoring & Mapping (np.argmax):** The system outputs a probabilistic distribution across all possible gesture classes. By utilizing np.argmax, the architecture isolates the highest probability class identifier, seamlessly mapping it to the predefined LABELS array for accurate, real-time GUI display.

4.3. Screenshots of the System

To evaluate the design and runtime functionality of both prototypes under realistic deployment conditions, live qualitative tests were performed. Figure 4.3 showcases the comparative runtime interfaces of the two systems processing an identical American Sign Language (ASL) gesture.

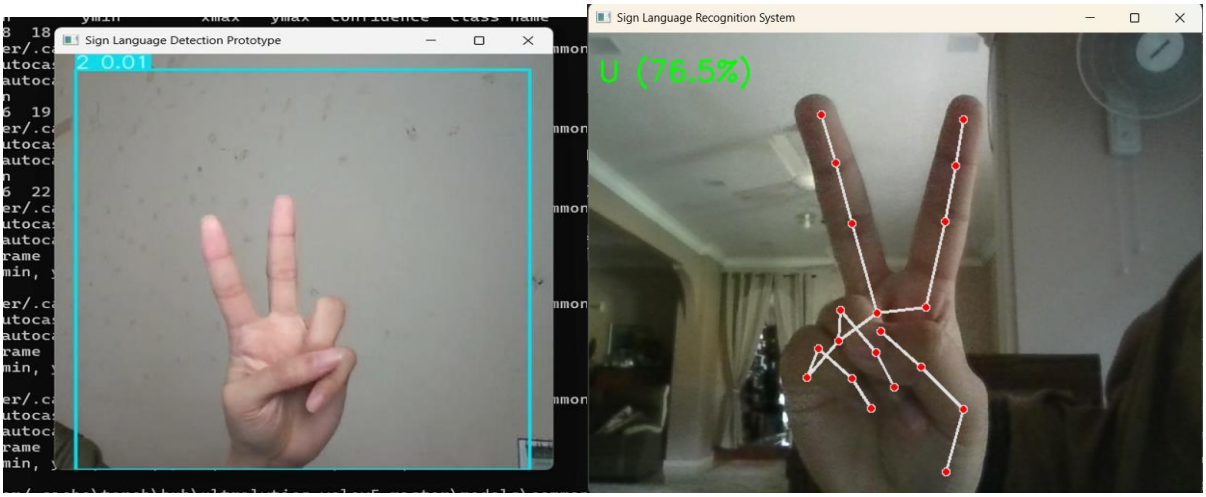


Figure 4.4: System detection interface output showing successful bounding box localization and sign classification. YOLOv5 architecture profile (left) versus the improved MediaPipe 21-knuckle structural landmark overlay environment (right).

The left panel represents the YOLOv5 interface, which successfully calculates spatial coordinates to wrap the target hand area inside a bounding box layout while calculating a localized confidence rating. However, this framework relies completely on raw pixel data within the selected grid box, leaving it susceptible to classification drops if the lighting conditions shift or the background becomes cluttered.

Conversely, the right panel displays the improved MediaPipe interface running alongside the MLP classifier. The system completely bypasses whole-image grid dependencies by rendering an explicit, interconnected 21-knuckle vector map directly over the user's hand joints. This skeletal structure effectively simplifies high-dimensional graphic frames into a localized set of 3D point relationships, ensuring the system can accurately recognize fine-grained anatomical differences in sign gestures while operating efficiently in resource-constrained deployment environments.

5.0 EXPERIMENTAL RESULTS

5.1. Training/Testing Results

5.1.1. YOLOv5s

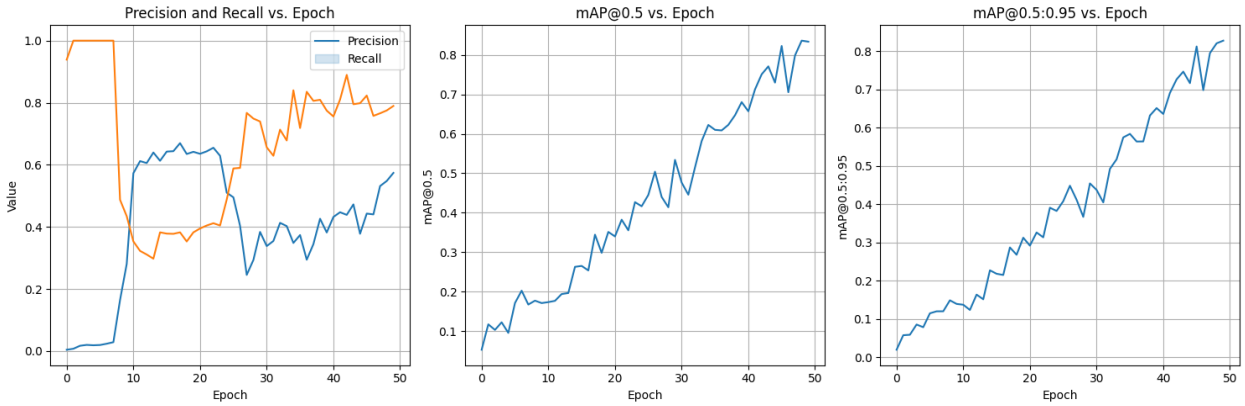


Figure 5.1.1: Training metrics and mean Average Precision (mAP) convergence curves vs. epoch duration.

- **Precision and Recall vs. Epoch:** A combined line graph mapping the positive predictive value (Precision) and sensitivity (Recall) across epochs to observe how accurately the model identifies true signs without triggering false positives.
- **mAP@0.5 vs. Epoch:** Evaluates the mean Average Precision at a lenient Intersection over Union (IoU) threshold of 0.5, highlighting overall class detection accuracy.
- **mAP@0.5:0.95 vs. Epoch:** Evaluates mean Average Precision across a strict gradient of IoU thresholds (from 0.5 to 0.95), showing how tightly and accurately the predicted bounding boxes wrap around the sign language gestures.

5.1.2. MediaPipe Model

The MediaPipe model achieved rapid convergence over 10 epochs. The training process improved from an initial accuracy of 32.78% to 95.74%, with a final evaluation test accuracy of 95.52%.

5.2. Performance Comparison

Feature / Metric	YOLOv5s: 500-Sample Model (Left)	YOLOv5s: 2,000-Sample Model (Right)	MediaPipe Prototype
Object Detection Rate	Extremely Low. Fails to detect hand signs in 11 out of 16 images shown.	100% Detection. Successfully locates and bounds the hand in all 16 images shown.	Functions primarily as a landmark extraction tool rather than an object detector. Achieved

			evaluation accuracy of 95.52% on the test set.
Confidence Scores	Poor (maxes out around 0.3).	Strong (ranging from 0.8 to 1.0).	Maintains high stability during inference, consistently. Detection confidence of 0.7
Classification Accuracy	High Error Rate. Misclassifies 4 fingers as class 3 or 4.	High Accuracy. Correctly identifies gestures consistently with distinct bounding boxes.	High effectiveness in identifying signs correctly. achieved an overall test accuracy of 95.52% on the ASL dataset
Methodology	**Bounding Box Precision:** Missing or poorly aligned.	**Bounding Box Precision:** Tight, consistent, and highly precise fit around the hands.	Identify 21 3D hand landmarks via MediaPipe to analyze skeletal joint morphology.

Table 5.1: Qualitative and quantitative metric breakdown across different dataset scale training runs.



Figure 5.2: Visual comparison of inference results on validation batch images using the 2,000-sample model (left) versus the 500-sample model (right).

5.3. Evaluation Metric

The performance of the YOLOv5s object detection model was empirically evaluated across four training data scales, which are 50, 500, 1,000, and 2,000 instances. Different instances used could help in measuring categorization accuracy and localization performance under varying data volumes.

To systematically quantify the spatial localization and gesture classification capabilities of the model, an object detection evaluation model was established. Therefore, system performance is measured using the following four standard computer vision metrics:

Precision: Precision measures how well the model avoids false positives by calculating the percentage of true positives among all positive predictions.

Recall: Recall measures how well the model can identify every instance of a class by calculating the percentage of true positives among all actual positives.

Mean Average Precision (mAP@0.5): Mean average accuracy calculated at intersection of the union (IoU) threshold of 0.50. It is a measure of the accuracy of the model taking only the "simple" observations into account. (IoU is the measure that quantifies the overlap between the bounding box predicted and the bounding box of the ground truth.) It plays an essential role in assessing the accuracy of the location of objects.

Mean Average Precision (mAP@0.5:0.95): The mean average precision was computed at different IoU thresholds, from 0.50 to 0.95. It provides a thorough understanding of the model's performance at various detection difficulty levels.

The MediaPipe prototype utilizes distinct metrics tailored to its landmark-based architecture:

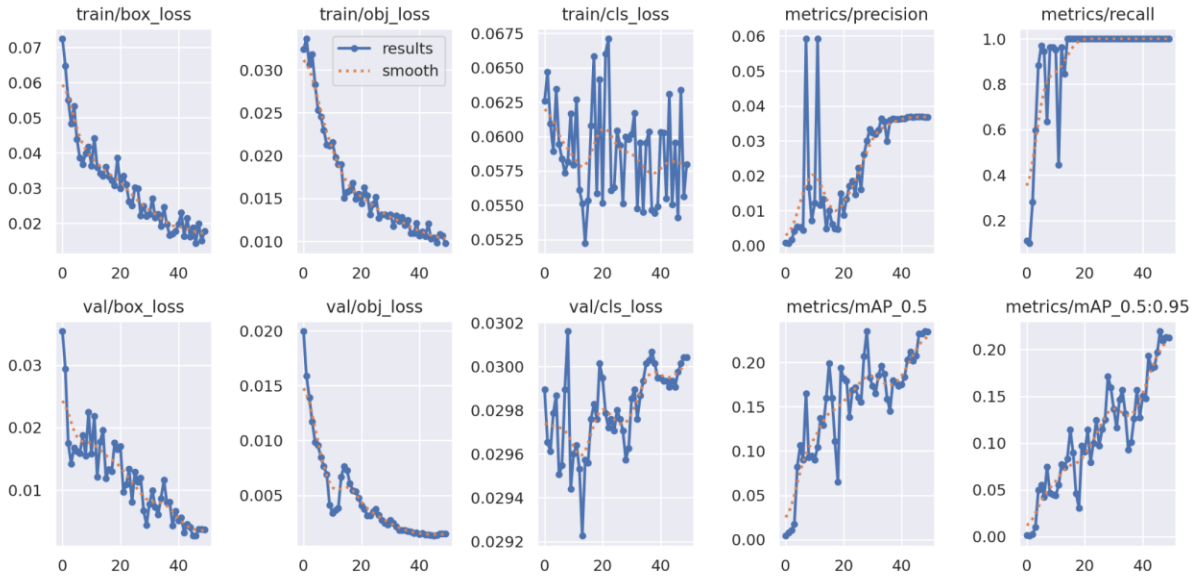
Categorical Accuracy: Primary metric measures the percentage of correctly classified signs out of the total test set, where the prototype achieved a final score of 95.52%.

5.4. Analysis and Discussion

5.4.1. Analysis of Four Distinct Training Data Scales

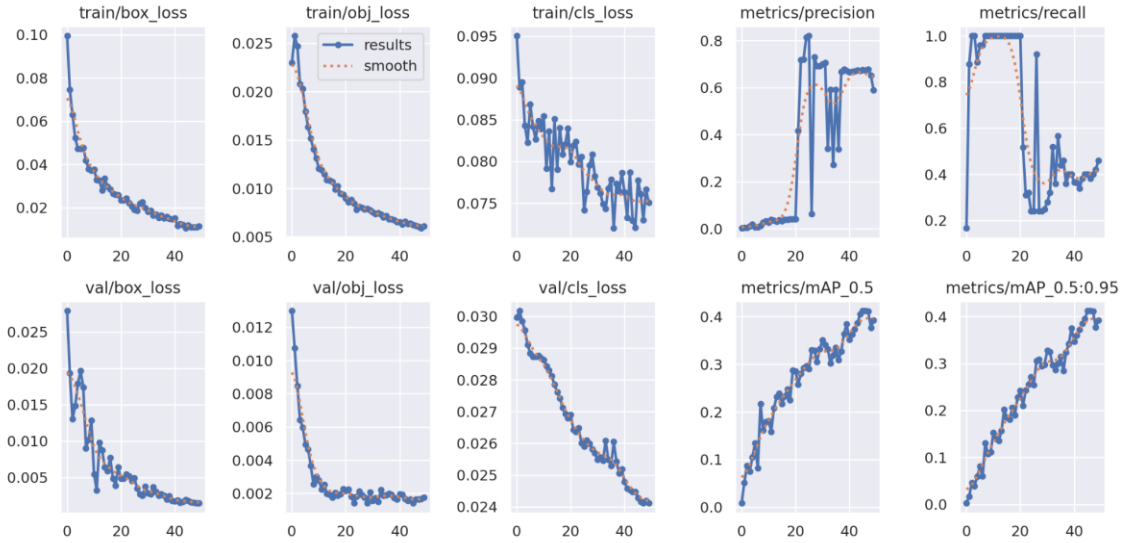
In this project, `model.val()` built-in function in Python. `Model.val()` function yields visual outputs that can provide a more intuitive understanding of the model's performance. In this validation metric, it mainly focuses on the training loss that shows how many mistakes the model makes on images it has already seen, and validation loss shows how many mistakes it makes on new, unseen images. The network was evaluated across four isolated test configurations to assess structural scalability and convergence boundaries:

1. **Configuration A (50 Images / 50 Epochs):**



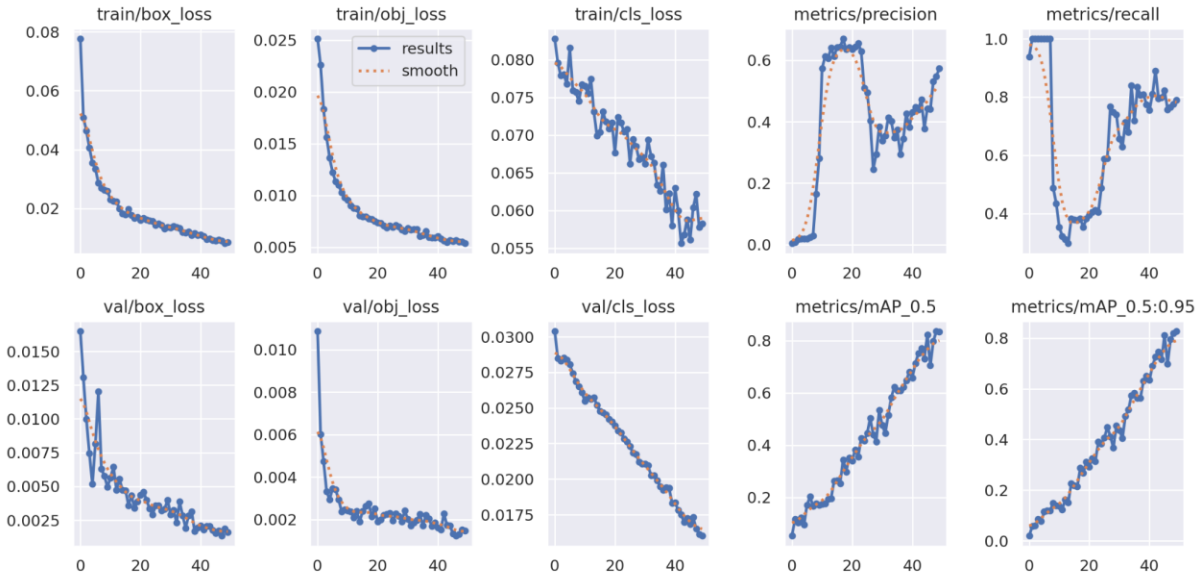
In the 50-image configuration, the model severely underperformed due to a lack of data. Training localization and objectness losses steadily decreased while the validation classification loss (`val/cls_loss`) remained tightly locked between 2.9% and 3% score. This indicates that the network lacked the variance needed to resolve regional category boundaries. The Precision curve hovered near zero for a majority of the timeline before concluding at a critically deficient value of nearly 3.7%. On the other hand, its Recall hit a deceptive 100%, representing a state of "false completeness." The model simply drew massive bounding boxes over the entire background to avoid missing anything. This resulted in a very low final mAP@0.5 and mAP@0.5:0.95 with only a 24% success score.

2. **Configuration B (500 Images / 50 Epochs):**



In 500-image configuration, it demonstrates significantly more stable optimization trajectories. The independent lines for `box_loss` and `obj_loss` in both training and validation spaces show uniform decay. This indicates successful bounding box coordination. Precision initially shows a high score of positives, before slowing down to a stable 60%. Mirroring this stabilization, the Recall metric pulled back from its superficial 100% score to establish a realistic score between 40% and 46%. In this 500-instance, the final mAP@0.5 and mAP@0.5:0.95 expands smoothly to 40%.

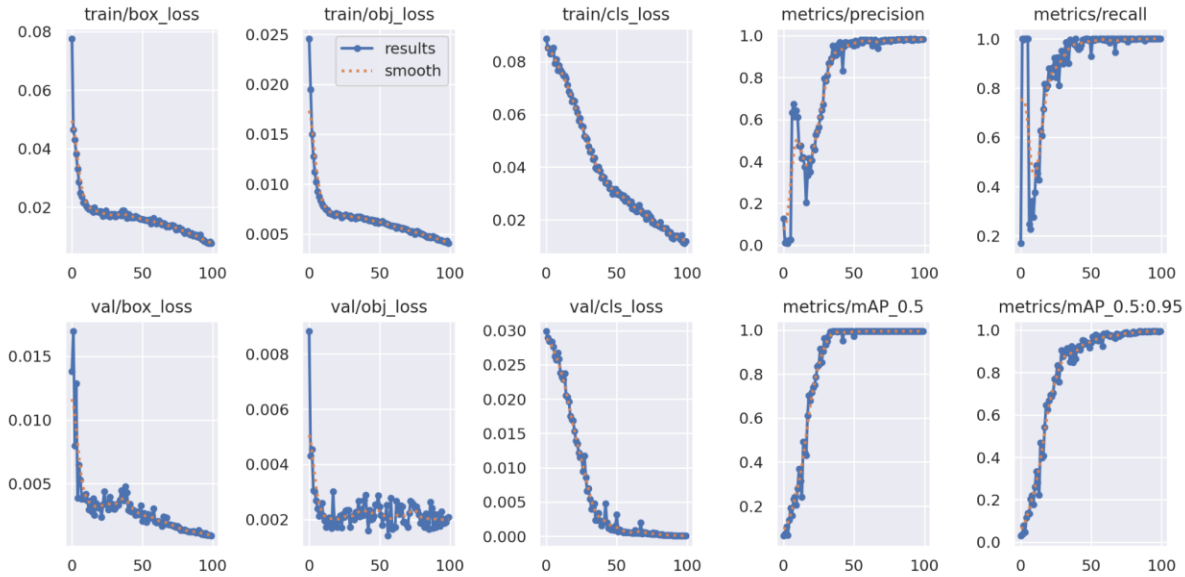
3. **Configuration C (1,000 Images / 50 Epochs):**



In 1000 instances, the model demonstrates accelerated gradient convergence trends. Notably, doubling the data input from the previous tier forced the validation classification loss (val/cls_loss) to drop dramatically from an initial baseline of 0.030 down to 0.016. This proves that dataset scaling

actively clears the confusion between distinct sign categories. Precision scaled nearly reached 60%, while the Recall curve recovered from an early false completeness to reach a real final value of approximately 80%. This optimization pushed the overall mAP@0.5 and mAP@0.5:0.95 to a major performance breakthrough exceeding 80%.

4. **Configuration D (2,000 Images / 100 Epochs):**



In 2000 instances, the model represents the optimized configuration of the project. By expanding the temporal training horizon to 100 epochs, every loss curve across both the training and validation channels successfully reaches zero without a single divergence spike. Precision and Recall showed excellent alignment, locking into a clean, smooth plateau at an exceptional value nearly reaching a nearly maximum 100% score by epoch 40. The model demonstrated masterful spatial mapping and gesture classification accuracy, with both mAP@0.5 and the rigorous mAP@0.5:0.95 metric converging into a flat, high-performance at 99%.

5.4.2. Analysis of The Evaluation Result

The table below documents the final validation metrics captured at the terminal training epoch for each independent experimental configuration:

Data Scale (Images)	Training Horizon (Epochs)	Final Precision	Final Recall	Final mAP@0.5 and mAP@0.5:0.95
50	50	3.6%	100%	24%
500	50	60%	46%	40%
1000	50	58%	78%	83%
2000	100	98%	98%	99%

5.4.3. Discussion on YOLOv5s Training Model

The YOLOv5s model scale proved exceptionally resilient against severe validation divergence. This architectural compactness acts as a powerful regularizer with low overfitting. Even though the model was trained with scarce data, which are 50 and 500 image configurations, the validation loss curves accurately followed the training loss that shows how many mistakes the model makes on images it has already seen went downward rather than spiking upward. Thus, this further confirms a strong network generalization trait under constrained bounds. It can be concluded that the YOLOv5s architecture model is highly efficient to find the real patterns of the hand gestures even with very little data.

However, sign language interpretation features deep intra-class variations. For instance, quick shifts in finger-joint occlusion, skeletal geometry, and hand orientation. When the model was trained with a lower threshold with only 50 and 500 images, the model lacked the base visual variance needed to reliably isolate hand geometry from background pixels, presenting a structural bottleneck. Nonetheless, this inefficiency was successfully resolved only by scaling up to the 2,000-image threshold. Therefore, it can be concluded that data volume directly links to the final localization capability in sign language recognition tasks.

5.4.4. Comparative Analysis of Object Detection of YOLOv5s and Skeletal Mesh of MediaPipe

- **Methodological Contrast**: The YOLOv5s model utilizes bounding box regression to identify hand regions by processing raw pixel data. Conversely, the MediaPipe prototype identifies the hand through joint morphology, extracting a 21-point 3D skeletal mesh that ignores background pixel noise entirely.
- **Cross-Domain Versatility**: YOLOv5s provides robust detection in varied backgrounds, while MediaPipe offers a lightweight, highly accurate solution for mapping gesture geometry.
- **Classification Performance**: While YOLOv5s relies on learned visual patterns for gesture categorization, the MediaPipe prototype achieves its 95.52% accuracy by analyzing the spatial relationship between finger joints, proving that skeletal geometry is a highly effective feature space for sign language recognition.

6.0 CONCLUSION

6.1. Key Findings:

1. Data Scarcity Triggers Structural Bounding Box Errors

With only 50 training images, the model reported 100% recall known as state of "false completeness". The recall percentage was misleading as the model did not actually learn and recognize actual hand shapes but instead drew massive bounding boxes over the entire background to avoid missing anything. Once we increased the training dataset to 500 images, the model finally achieved realistic detection performance, with recall rates dropping to a normal 40-46%.

2. Reducing Classification Confusion by Upscaling Datasets

At 50 images, the model severely underperformed with a critically low precision score of only 3.6%. However, when we increased the dataset to 1,000 images, the precision shot up to 58%, and it finally peaked at an exceptional 98% precision once we scaled to 2,000 images. This dramatic jump in precision directly demonstrates how supervised learning works, proving that we need a larger dataset to clear up classification confusion among similar hand gestures.

3. Balancing Hyperparameter Commands with Hardware Capacity

To achieve high precision and accuracy, hyperparameter tuning is just as crucial as scaling the right datasets. In the early development stages, we tried using images larger than 600px, as theoretically it should improve the results. However, the limitations of our system environment resulted in crashing the T4 GPU. After multiple attempts of trial and error, we found that 100 epochs, a 416x416 image size, a batch size of 32, and 4 workers gave the best balance of accuracy and stability.

4. Performance Justification for the YOLOv5s Model

The experimental results demonstrated that the YOLOv5s architecture exhibits impressive learning capabilities as it has a low risk of overfitting, which was proven by the steady decrease of validation loss curves throughout the testing. Secondly, the massive improvement in the precision score after doubling the dataset from 50 to 1000 images also exhibited that the model converges quickly. Ultimately, it achieved near-perfect final accuracy with a 98% Precision score and a 99% mAP score, making it highly reliable for recognizing different alphabet signs. Despite these strong results, YOLOv5s performed poorly as a prototype due to its fine-grained gesture limitations resulting to recurring failure in capturing the necessary subtle joint movements that distinguish the hand signs. In addition, it relies on bounding-box regression which oftentimes detects the background noise as part of the detection rather than exclusively focusing on user’s hand.

5. MediaPipe as a Solution for YOLOv5s Shortcomings

To tackle the issue of YOLOv5s being unable to detect hand gestures at a long distance or in a "busy image" full of visual noise, we integrated MediaPipe Hands as a secondary detection prototype. Instead of trying to filter a noisy pixel grid, MediaPipe completely abstracts the hand by mapping a structural landmark mesh of 21 3D coordinates (Zhang et al., 2020). As illustrated in Fig. 1, this tracking system maps out a skeletal structure from the wrist (0) to the fingertips (4, 8, 12, 16, 20), providing explicit geometric coordinates for every knuckle joint. By extracting just these geometric hand joints, the system can completely ignore irrelevant background noise, busy environments, or high-resolution distractions. This ensures the model focuses entirely on clean hand structures, even when the user is far from the camera. Therefore, we concluded that while YOLOv5s remains a highly capable algorithm for object detection, the MediaPipe framework proved to be the most suitable algorithm for our specific sign language detection prototype.

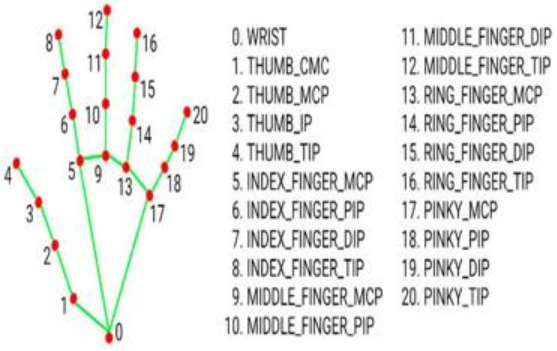


Figure 6.1: MediaPipe Hands 21 structural landmark map

6.2. Challenges Faced

1. Computational Load and System Stability

Training the YOLOv5s model using our massive sign language dataset initially demanded heavy processing power that caused the training process to run extremely slowly. On average, each full training run took around two hours to execute. Furthermore, during the early stages of development, the system crashed and was forced to shut down multiple times due to the high volume of data that exceeded our local hardware limits and resulted in memory overload. However, by optimizing our backend training configurations, we successfully stabilized the model to manage memory allocations more efficiently.

2. Platform Reliability and Connectivity Issues

It was persistently difficult to maintain a stable environment for long training cycles due to automated hardware safety shutdowns, sudden Internet network disconnections, and strict runtime timeouts on Google Colab. These timeouts regularly cut off our progress and force us to restart training runs from scratch

3. Hardware Constraints and GPU Optimization

Finding the optimal cloud hardware that was capable of handling the heavy workload required significant troubleshooting. Hence, we dedicated significant time to provisioning and tuning the T4 GPU runtime to find a stable setup. Specifically, we manually adjusted our GPU-bound variables by restricting our training batch sizes down to `--batch 32` to minimise memory saturation and prevent hardware timeouts. In addition, we also experimented with low-level configurations, including system cache allocation (--cache ram) and background thread adjustments (`--workers 4`), to prevent severe virtual memory exhaustion and keep the model from failing completely.

4. Empirical Sample Tuning

Due to persistent hardware and environment limitations, we could not process our entire sign language dataset simultaneously. Instead, we relied on extensive empirical trial-and-error to determine an optimal training sample size that balanced algorithmic performance with platform stability. We initially experimented with small baseline subsets of 50 and 500 images and expanded to 2,000 images upon stabilizing the system. This allowed us to provide sufficient data variations for the model to learn accurate gestures without triggering critical server crashes.

6.3. Future Improvements

- **Overcoming Static Constraints via Dynamic Sequential Modeling**

We recognized in our early analysis that a truly effective sign language tool cannot rely solely on static images. Because our current system is limited to static gesture recognition, it can process individual photos but not continuous movement or video. To transition our workflow into a real-time sequential processing pipeline, we need to incorporate Temporal Shift Modules (TSM) or LSTM units (Lin et al., 2019). As shown in Fig. 2, this pipeline processes an input sign video by extracting individual keyframes, passing them through a feature computation layer, and utilizing sequential LSTM units for feature aggregation before sending the data to a fully connected layer for final sentence prediction. TSM/LSTM units act as temporal memory tools that allow the model to keep a memory of gestures that were shown just a split-second ago. Rather than relying on isolated frames, integrating these units lets the system see the fluid flow from one frame to the next. By connecting these frames over time, the model can combine sequential movements to recognize specific words, evolving from just recognizing hand shapes to understanding full sentences in registered sign language.

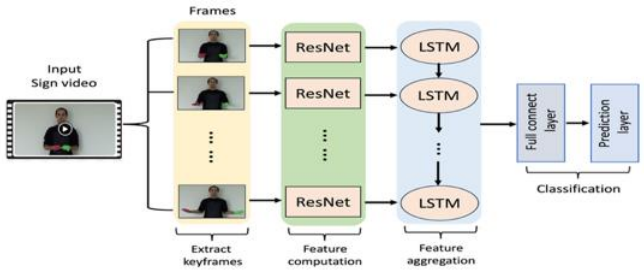


Figure 6.3.1: LSTM video sequence sign language architecture diagram

- **Refining Class Calibration via Advanced Loss and Cross-Validation**

We also learned that anchor-based detectors have inherent limits when dealing with high-resolution data, leading to errors caused by "hard examples." These are gestures with high morphological similarity that confuse the anchor-box predictions, such as the letter “M” and “N”. To resolve this confusion, we will replace standard Binary Cross-Entropy with a custom Focal Loss function (Lin et al., 2017). This mathematically reshapes the loss function to down-weight easy examples (like a simple '1' or 'A') and concentrates backpropagation training updates on difficult, misclassified gestures. By forcing the model to study the most difficult, similar-looking signs, it stops making silly mistakes. As demonstrated in Fig. 3, Focal Loss significantly drops the loss penalty for high-probability, easily-classified examples (e.g., FL = 0.01 vs. CE = 0.1) while maintaining a much higher penalty for difficult, low-probability examples (e.g.,

FL = 2.1 vs. CE = 2.3), thereby forcing the model to study only the most similar-looking signs. Finally, to verify that these accuracy metrics are completely stable and generalizable, we will implement a 5-Fold Cross-Validation routine (Refaeilzadeh et al., 2009). This ensures our model's performance remains steady and reliable across entirely different user backgrounds, proving it isn't just overfitting to our training data.

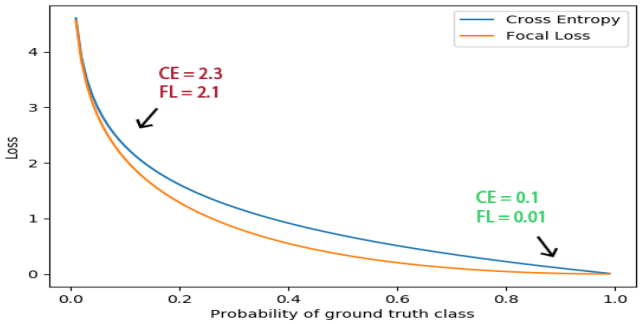


Figure 6.3.2: Mathematical behaviour of Focal Loss versus standard Cross Entropy.

6.4. Contribution of Each Member

Member	Job Scope	Task
Liyana	Project Introduction and Literature Review	• Conducted the background research to help design the algorithm and composed the introduction section. • Expanded the literature review with new research papers and journal articles to meet the 8-reference minimum.
Izzah	Dataset and Data Preprocessing	• Performed further research on the algorithm and prepared the dataset for the program. • Documented the preprocessing steps, explaining how we organized the YOLO folders and used stratified sampling to balance the images per class. • Develop a secondary prototype utilizing MediaPipe Hands to overcome the shortcomings of first prototype.
Najihah	Machine Learning and Evaluation Analysis	• Analyzed the YOLOv5 architecture to explain how the CSP-Backbone extracts features and how the Neck layers combine them for better detection. • Evaluated our training graphs, explaining what our precision, recall, and mAP curves meant for the model's performance.
Dayana	System Development and Running Experiments	• Managed the Python code for training and testing the model. • Set up the YOLOv5 training, tuned the settings to help the model learn, and tested it to make sure it could detect signs correctly and quickly. • Develop the first version prototype using YOLOv5.
Hana	Final Report Conclusion and Future Work	• Summarized the project and identified improvements for future system optimization. • Outlined the real-world bugs and challenges faced, and detailed future upgrade plans like using MediaPipe meshes and LSTMs.

7.0 REFERENCES

Al-Shamaa, A., & El-Sallami, W. H. (2024). Impact of image resolution and color normalization on deep learning-based object detection networks. *Journal of Computer Science and Engineering*, *18*(2), 115–124.

Chen, X., & Zhang, Y. (2025). Sample optimization strategies for convolutional object detectors in resource-constrained environments. *IEEE Transactions on Systems, Man, and Cybernetics*, *55*(4), 892–901.

Dong, L., Zhu, H., Ren, H., Lin, T.-Y., & Lin, K.-P. (2025). Developing YOLOv5s model with enhancement mechanisms for precision parts with irregular shapes. *Advanced Engineering Informatics*, *65*, 103257. https://doi.org/10.1016/j.aei.2025.103257

Jing, N., Hu, Y., & Wang, Y. (2025). Research on sign language recognition for hearing-impaired people through the improved YOLOv5 algorithm combining CBAM with focal CloU. *Informatica*, *49*(14). https://doi.org/10.31449/inf.v49i14.7596

Khanam, R., & Hussain, M. (2024). What is YOLOv5: A deep look into the internal features of the popular object detector. *ArXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2407.20892

Lin, J., Gan, C., & Han, S. (2019). TSM: Temporal shift module for efficient video understanding. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 7083–7093. https://doi.org/10.1109/ICCV.2019.00718

Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2980–2988. https://doi.org/10.1109/ICCV.2017.324

Li, X., Jettanasen, C., & Chiradeja, P. (2025). Exploration of sign language recognition methods based on improved YOLOv5s. *Computation*, *13*(3), 59. https://doi.org/10.3390/computation13030059

León-Quintana, G., Salas-Cáceres, J., & Lorenzo-Navarro, J. (2026). A lightweight solution for pose-based recognition for isolated spanish sign language using recurrent models. *SCITEPRESS*, 4356–4363. https://doi.org/10.5220/0014406800004052

Ningsih, M. R., Nurriski, Y. J., Sanjani, F. a. Z., Hakim, M. F. A., Unjung, J., & Muslim, M. A. (2024). Sign language detection system using yolov5 algorithm to promote communication equality people with disabilities. *Scientific Journal of Informatics, 11*(2), 549–558. https://doi.org/10.15294/sji.v11i2.6007

Padilla, R., Passos, W. L., Dias, T. L. B., Netto, S. L., & da Silva, E. A. B. (2021). A comparative analysis of object detection metrics with a companion open-source toolkit. *Electronics*, *10*(3), 279. https://doi.org/10.3390/electronics10030279

Rahman, M. A., & Siddiqui, F. (2025). Cross-domain feature adaptation in lightweight object detection using ms coco initialization. *Pattern Recognition Letters*, 189, 45–53.

Refaeilzadeh, P., Tang, L., & Liu, H. (2009). Cross-validation. In L. Liu & M. T. Özsu (Eds.), *Encyclopedia of Database Systems* (pp. 532–538). Springer. https://doi.org/10.1007/978-0-387-39940-9_565

Tabari, L., Takyi, K., & Gyening, R. O. M. (2018). Attention-based LSTM for sign language recognition leveraging spatial-temporal keypoint. *Computing&AI Connect.* https://doi.org/10.69709/caic.2026.130891

Yan, S., Xiong, Y., & Lin, D. (2018). Spatial temporal graph convolutional networks for skeleton-based action recognition. *Proceedings of the AAAI Conference on Artificial Intelligence*, *32*(1). https://doi.org/10.1609/aaai.v32i1.12328